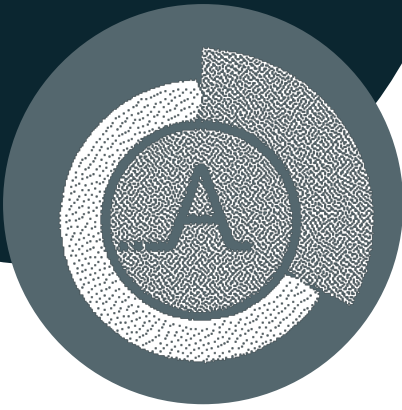




Rapport d'activité: Affluences

Première année d'alternance

Maître d'apprentissage: Luis Valdez

Tuteur Enseignant: Faten Atigui

	Remerciements	3
	Introduction	4
	Affluences	4
	Environnement de Travail	5
	Conditions de travail et intégration	5
	Conclusion partielle	13
	Missions	14
	Optimisation de requête	14
	Création du app-service	20
	Annexes	24
	Rona : automatiser les commits	24
	Rédaction avec Typst et clean-cnam-template	26
	Glossaire	27

Remerciements

Avant de commencer la lecture de ce mémoire, je tiens à adresser mes sincères remerciements aux personnes qui ont contribué au bon déroulement de mon année d'alternance et à la réalisation de ce document.

Je souhaite tout d'abord remercier l'entreprise Affluences pour m'avoir accueilli. J'ai particulièrement apprécié l'environnement de travail agréable et la liberté qui m'a été accordée dans le choix de mes outils de développement, me permettant de travailler dans des conditions optimales. Je remercie également l'ensemble de mes collègues pour leur accueil et la bonne ambiance générale.

Mes remerciements s'adressent tout particulièrement à mon maître d'apprentissage, Luis Valdes. Sa disponibilité constante, ses conseils avisés et son accompagnement m'ont été précieux tout au long de l'année. Son management bienveillant, alliant confiance et soutien, m'a permis de m'épanouir tant dans l'entreprise que dans mes missions.

Je remercie également Micaël Pais Novo, CTO, pour sa disponibilité et la confiance qu'il m'a témoignée, me permettant de travailler en autonomie tout en sachant que je pouvais compter sur son aide.

Enfin, je souhaite exprimer ma gratitude à mes collègues pour leur aide précieuse. Merci à Jean-Charles Moussé pour son soutien sur le projet `app-api`, à Raphaël Galmiche pour son aide sur les déploiements, et à Justine Ribas pour ses éclaircissements sur `data-service`.

Introduction

Ce rapport dresse le bilan de mon expérience en entreprise au cours de l'année scolaire 2025-2026. Ce parcours s'inscrit dans le cadre de ma formation d'ingénieur en informatique et systèmes d'information, réalisée en alternance au sein de l'École d'Ingénieur du Conservatoire National des Arts et Métiers ([EI-CNAM](#)).

J'ai donc intégré en alternance l'équipe [Backend](#) d'[Affluences](#), une entreprise française innovante spécialisée dans la gestion de l'affluence et l'optimisation des flux de visiteurs. Cette année représentait pour moi une toute nouvelle aventure : mes premiers mois au sein de la société, une immersion complète dans un environnement professionnel exigeant et stimulant. J'intègre la partie **Internal Services** en tant que **développeur** [Backend](#).

I - Affluences

[Affluences](#) est une entreprise française fondée en 2014, aujourd'hui leader européen de la mesure et de la prévision d'affluence en temps réel. Avec plus de **10 ans d'expertise**, sa mission est de transformer la gestion des flux de visiteurs en une expérience fluide et optimisée, tant pour les établissements que pour leurs usagers.

L'entreprise affiche des résultats impressionnants : **1 800 établissements clients** répartis à travers l'Europe, une application mobile notée **4,8/5** utilisée par plus d'**un million de personnes**, et **13 millions de consultations mensuelles**. Elle a réalisé une levée de fonds de 4 millions d'euros en 2020 et compte parmi ses clients des institutions de renom comme le Musée du Louvre, la Tour Eiffel, la SNCF et l'Université de Cambridge.

Sa force réside dans une solution technologique complète et intégrée, qui combine des capteurs [IoT](#) propriétaires pour la collecte de données, des algorithmes prédictifs pour anticiper les pics d'activité, et des plateformes de communication multi-canaux pour informer les utilisateurs en temps réel. L'entreprise déploie ses solutions sur **huit secteurs verticaux** distincts : bibliothèques et médiathèques, musées et lieux culturels, universités et smart campus, collectivités et smart cities, transports publics, espaces naturels, entreprises et smart buildings, retail et événements.

L'équipe technique d'une cinquantaine de collaborateurs est organisée en pôles spécialisés : **Data** (traitement des flux de données en temps réel), **Computer Vision** (algorithmes d'[Internet of Things](#) avec intelligence artificielle), **Infra** (infrastructure et sécurité), **Web** (interfaces utilisateur), et **Service** ([APIs](#) et **microservices**). L'entreprise s'appuie sur un stack technologique moderne incluant [Node.js](#) et [NestJS](#) pour le [Backend](#), [Kafka](#) pour le streaming de données, [Airflow](#) et [Argo Workflow](#) pour l'orchestration des workflows, [Kubernetes](#) pour l'orchestration de conteneurs, et [Datadog](#) pour le monitoring des applications. L'équipe a récemment adopté une architecture [Monorepo](#) pour certains projets, améliorant la modularité et la maintenance. L'organisation suit une méthodologie [Agile](#) avec des sprints de 2 semaines.

C'est au sein de cette [Scale-up](#) innovante, qui promeut une culture d'autonomie et d'actionnariat salarié universel, que j'ai eu l'opportunité de réaliser mon alternance.

Environnement de Travail

Mon alternance s'est déroulée au sein d'un environnement de travail stimulant, caractérisé par une forte culture d'entreprise et une organisation agile et moderne. Cette section détaille les conditions de travail, l'environnement technique, et les méthodologies qui régissent le quotidien au sein de la société.

I - Conditions de travail et intégration

L'environnement de travail au sein de l'entreprise se distingue par une culture fondée sur la confiance, l'autonomie et la prise d'initiative. Avec une équipe d'une cinquantaine de personnes, l'organisation conserve une hiérarchie aplatie qui favorise la communication directe et la collaboration.

Un aspect particulièrement marquant est son modèle d'actionnariat salarié universel : chaque employé est actionnaire, ce qui aligne les intérêts de tous sur le succès collectif de l'entreprise. La transparence est également une valeur clé, avec une communication ouverte sur les résultats et la stratégie de l'entreprise.

L'intégration des nouveaux arrivants, et notamment des alternants, est facilitée par un système de mentorat et des perspectives d'évolution interne concrètes, illustrées par des parcours comme celui du Lead Mobile, qui a débuté en tant que stagiaire.

Environnement technique et outils

Son écosystème technique est riche et moderne, conçu pour supporter une plateforme traitant des millions d'utilisateurs et des centaines de millions de points de données annuellement.

Stack technique principale

L'architecture [Backend](#) repose sur une approche **microservices**, utilisant principalement [Node.js](#) et **Python** pour le traitement des données. La communication asynchrone entre les services est assurée par [Kafka](#). Côté [Frontend](#), les applications web s'appuient sur [Angular](#), tandis que l'application mobile a été développée avec [Flutter](#), le framework cross-platform de Google.

Langages et frameworks

L'équipe de développement maîtrise un large éventail de langages et frameworks pour répondre aux besoins spécifiques de chaque partie de la plateforme :

- **Backend** : [Node.js](#), Python.
- **Frontend** : JavaScript/[TypeScript](#) avec [Angular](#), [GraphQL](#) pour les [APIs](#).
- **Mobile** : Dart avec [Flutter](#), avec une expérience passée sur le natif (Swift/Kotlin).

Infrastructure et déploiement

L'infrastructure est hébergée sur le [Cloud](#) français OVHcloud pour garantir la conformité [RGPD](#). L'architecture distribuée s'appuie sur la conteneurisation avec [Docker](#), probablement orchestrée par [Kubernetes](#). Les bases de données suivent une approche multi-modèle, combinant probablement des bases de données relationnelles ([SQL](#)), [NoSQL](#) (pour les séries temporelles des capteurs) et un cache en mémoire comme [Redis](#) pour les données temps réel.

Architecture logicielle : Clean Architecture

Tous les **microservices** développés en interne suivent un pattern **Clean Architecture** rigoureux, qui impose une séparation stricte des responsabilités entre les couches logicielles. L'objectif est d'isoler la logique métier de toute dépendance technique (base de données, framework HTTP, messagerie), rendant le code testable et évolutif indépendamment de son infrastructure.

Structure en couches

Chaque service est organisé selon une arborescence de modules reproductible :

```

1 src/
2   └─ core/
3       └─ repositories/
4           └─ device.repository.ts          # Interface IDeviceRepository
5   └─ modules/
6       └─ devices/
7           └─ entities/                    # Entités TypeORM (mapping BD)
8               └─ device.entity.ts
9           └─ models/                      # Modèles métier internes
10              └─ device.model.ts
11          └─ adapters/                    # Conversion entité ↔ modèle
12              └─ device.adapter.ts
13          └─ repositories/                # Implémentation MySQL
14              └─ device-mysql.repository.ts
15          └─ services/                    # Logique métier pure
16              └─ device.service.ts
17          └─ controllers/                  # Exposition REST
18              └─ device.controller.ts

```

Abstraction des repositories

Les interfaces de repository sont définies dans `core/`, sans aucune dépendance à un moteur de base de données. La couche service ne connaît que ces contrats :

```

core/repositories/device.repository.ts

1 export interface IDeviceRepository {
2   findById(deviceId: number): Promise<Device | null>;
3   findByApiKey(apiKey: string): Promise<Device | null>;
4   save(device: CreateDeviceParams): Promise<Device>;
5 }

```

L'implémentation concrète, qui dépend de TypeORM et MySQL, vit dans `repositories/` et implémente ce contrat :

```

modules/devices/repositories/device-mysql.repository.ts

1 @injectable()
2 export class DeviceMysqlRepository implements IDeviceRepository {
3   constructor(
4     @inject(DEVICE_ENTITY_REPO) private repo: Repository<DeviceEntity>,
5   ) {}
6
7   async findById(deviceId: number): Promise<Device | null> {
8     const entity = await this.repo.findOne({ where: { deviceId } });
9
10    return entity ? DeviceAdapter.toDomain(entity) : null;
11  }
12 }

```

Cette indirection permet de substituer l'implémentation MySQL par une implémentation en mémoire lors des tests unitaires, sans modifier une seule ligne de la logique métier.

Ségrégation des types : Entités, Modèles et DTOs

Une distinction rigoureuse est maintenue entre trois catégories de types qui ne doivent jamais se mélanger :

Definition 3.1. (Trois catégories de types):

- **Entités (ORM^s)** : reflètent fidèlement la structure de la base de données. Décorées avec les annotations TypeORM (`@Entity` , `@Column`), elles ne sortent jamais de la couche infrastructure.
- **Modèles métier** : types internes au `Microservices`, indépendants de tout moteur de persistance. Ce sont eux qui transitent dans la couche service et définissent le langage ubiquitaire du domaine.
- **DTOs^s** : types à la frontière de l'`APIs`. Définis pour les corps de requête et les réponses REST, ils sont décorés pour la validation (`class-validator`) et la documentation Swagger. Ils ne contiennent aucune logique métier.

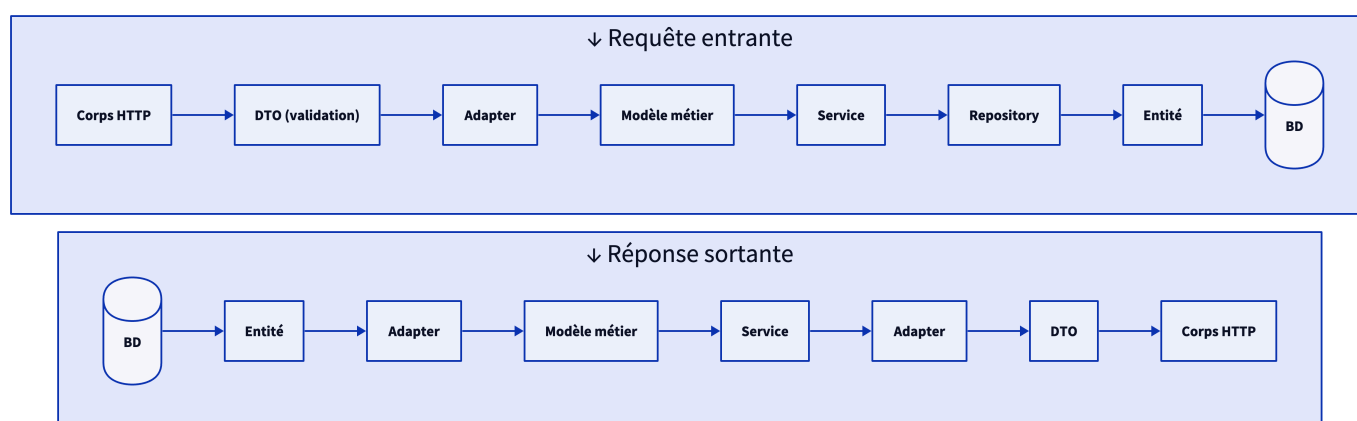


Fig. 1 - Flux de données à travers les couches

Adapters

Les adapters sont des classes utilitaires statiques qui assurent la conversion bidirectionnelle entre les couches. Ils constituent la seule couche du code où deux types de représentations différents se côtoient :

```

1 export class DeviceAdapter {
2   static toDomain(entity: DeviceEntity): Device {
3     return {
4       deviceId:    entity.deviceId,
5       identifi er: entity.identifi er,
6       apiKey:     entity.apiKey,
7       isBlacklisted: entity.blacklist,
8       revokedAt:   entity.revokedAt ?? null,
9       revokedReason: entity.revokedReason ?? null,
10    };
11  }
12
13  static toResponseDto(model: Device): DeviceResponseDto {
14    return {
  
```

```

15         id:          model.deviceId,
16         identifier: model.identifier,
17         revoked:     model.revokedAt ≠ null,
18     };
19 }
20 }

```

Ce pattern garantit que la forme des données en base de données ne dicte jamais la forme des objets métier, et vice-versa. Chaque couche peut évoluer indépendamment : renommer une colonne en base n'impacte que l'entité et son adapter, jamais la logique service ni les [DTOs](#)®.

Bénéfices observés

En pratique, cette architecture apporte trois avantages concrets au sein des équipes :

Exemple 3.1. (Apports de la Clean Architecture):

1. **Testabilité** : les services peuvent être testés unitairement en injectant un repository en mémoire, sans base de données réelle.
2. **Indépendance technologique** : passer de MySQL à PostgreSQL pour un service revient à écrire une nouvelle implémentation du repository, sans toucher à la logique métier.
3. **Lisibilité** : un développeur qui rejoint le projet sait immédiatement où se trouve chaque type de code grâce à la structure prévisible des modules.

Outils de productivité

Conformément à la culture d'autonomie, les développeurs ont la liberté de choisir leurs outils de travail, que ce soit leur système d'exploitation (OS) ou leur environnement de développement intégré (IDE), leur permettant de travailler dans des conditions de confort optimales.

Sécurité et accès aux ressources

La sécurité est une priorité absolue. La plateforme est entièrement conforme au [Reglement General sur la Protection des Donnees](#)®, avec des mesures strictes d'anonymisation des données, de limitation de la durée de conservation et de chiffrement ([SSL/TLS](#)®). En complément de cette conformité, l'entreprise est actuellement en démarche pour obtenir la certification [ISO 27001](#)®, la norme internationale de référence pour les systèmes de management de la sécurité de l'information, afin de formaliser et d'attester de la robustesse de ses processus.

Observabilité et monitoring

Le suivi de la plateforme en production est assuré par plusieurs outils. [Sentry](#)® est utilisé pour le tracking d'erreurs en temps réel. Une solution d'[Application Performance Monitoring](#)® et un système de logging centralisé sont également en place pour superviser la performance des **microservices** et faciliter le débogage.

Outils collaboratifs

La collaboration est facilitée par la structure plate de l'entreprise et l'utilisation d'outils de communication modernes. Les réunions régulières comme les « Moments Affluences » et la transparence générale sur les objectifs permettent à chacun de comprendre sa contribution à la vision globale.

Gestion des déploiements et workflow Git

Les déploiements sont automatisés via un pipeline de [Integration Continue / Deploiement Continu \(CI/CD\)](#) et s'appuient sur un workflow Git structuré qui régit aussi bien le développement quotidien que le processus de publication des versions.

Branches de fonctionnalité

Tout développement (nouvelle fonctionnalité, correction de bug ou refactoring) fait l'objet d'une branche dédiée créée depuis `main`. La convention de nommage suit le format `<type>/<description-courte>`, et inclut systématiquement le numéro de ticket ou d'épic Jira associé, par exemple :

```
feat/app-service_first-version_INT-3713
feat/INT-1234_app-service
fix/attendance-stats-timeout_INT-987
refactor/device-repository-abstraction_INT-1056
```

La présence du ticket dans le nom de branche permet aux outils d'intégration (Jira, GitLab) de lier automatiquement la branche à son ticket, et offre une visibilité immédiate sur le contexte de chaque développement sans avoir à consulter l'historique des commits.

Une fois les développements terminés, une **Pull Request** est ouverte pour une revue de code par au moins un autre développeur avant fusion dans `main`. Cette pratique garantit la cohésion du code et le partage de connaissances au sein de l'équipe.

Commits conventionnels

Tous les commits doivent respecter la spécification **Conventional Commits**. Le format impose un type, un périmètre optionnel et une description courte : `<type>(<périmètre>): <description courte>`

Exemple 3.2. (Exemples de commits):

```
feat(devices): add PATCH endpoint for partial device update
fix(attendance): resolve timeout on 30-day period queries
chore(deps): bump @affluences/commons to 2.4.1
refactor(app-versions): extract pagination to shared utility
test(devices): add unit tests for DeviceAdapter
ci: update GitLab pipeline to Node 20
```

Les types principaux reconnus sont `feat` (nouvelle fonctionnalité), `fix` (correction de bug), `chore` (maintenance), `refactor`, `test`, `docs` et `ci`.

Chaque commit référence également le ticket Jira correspondant, ajouté en fin de description avec le préfixe `#`. Le projet Jira de l'équipe **Internal Services** utilise le préfixe `INT` :

Exemple 3.3. (Référencement du ticket Jira):

```
feat(devices): add PATCH endpoint for partial device update #INT-1234
fix(attendance): resolve timeout on 30-day period queries #INT-987
chore(deps): bump @affluences/commons to 2.4.1 #INT-1056
```

Cette pratique assure la traçabilité bidirectionnelle entre le code et les tickets : depuis l'historique git, on retrouve le contexte fonctionnel de chaque changement ; depuis Jira, on accède directement aux commits et aux pull requests associés.

Ces règles ne reposent pas sur la bonne volonté des développeurs : elles sont **mécaniquement enforced** par un hook pre-commit via **commitlint**. Le fichier `.commitlintrc.json` à la racine du dépôt définit les contraintes :

```

1 {
2   "rules": {
3     "scope-empty": [2, "never"],
4     "type-enum": [2, "always", [
5       "feat", "fix", "docs", "refactor",
6       "test", "revert", "quality", "chore", "init"
7     ]],
8     "subject-case": [0]
9   },
10  "parserPreset": {
11    "parserOpts": {
12      "headerPattern": "^(\\w*)(?:\\(((\\w*)\\))?:\\s(\\.*)\\s([A-Z]+-[0-9]+)$",
13      "headerCorrespondence": ["type", "scope", "subject", "ticket"]
14    }
15  }
16 }
```

La `headerPattern` est la pièce centrale : elle valide que chaque message respecte le format `<type>(<scope>): <description> <TICKET-ID>`, et extrait les quatre composants (`type`, `scope`, `subject`, `ticket`) pour les outils en aval. Un commit sans scope ou sans référence de ticket est **rejeté** avant même d'être créé.

Cette convention rend l'historique git directement lisible et sert de base à la génération automatique des changelogs lors des publications.

Branches de release et publication avec release-it

Le processus de publication s'appuie sur des **branches de release** dédiées, nommées `release/<version>` (ex : `release/1.2.3`). Ces branches concentrent tous les commits de publication générés automatiquement par l'outil **release-it**, qui orchestre l'ensemble du cycle de vie d'une version.

Phases de publication d'une version

Phase 1 : Release Candidate

Une première version candidate est générée depuis la branche de release (`1.2.3-rc.0`). **release-it** met à jour les fichiers `package.json`, génère un `CHANGELOG` partiel depuis les commits conventionnels et crée le tag git `v1.2.3-rc.0`. Cette RC est déployée en environnement de staging pour validation fonctionnelle.

Phase 2 : Release définitive

Après validation, la version finale `1.2.3` est publiée. **release-it** génère le `CHANGELOG` complet, crée le tag `v1.2.3`, publie le package sur le registry npm interne de la société, puis la branche `release/1.2.3` est fusionnée dans `main`.

release-it orchestre automatiquement les étapes suivantes à chaque publication :

- Validation que le dépôt est propre (pas de modifications non commitées)
- Calcul du prochain numéro de version selon les règles du [Semantic Versioning](#)^a et des commits conventionnels
- Mise à jour de `package.json` et `package-lock.json`
- Génération ou mise à jour du fichier `CHANGELOG.md`
- Création du commit de release et du tag git signé
- Publication sur le registry npm interne

Ce processus garantit une traçabilité complète des livraisons : chaque version déployée correspond à un tag git précis, et son contenu est documenté dans le changelog généré automatiquement depuis les commits conventionnels.

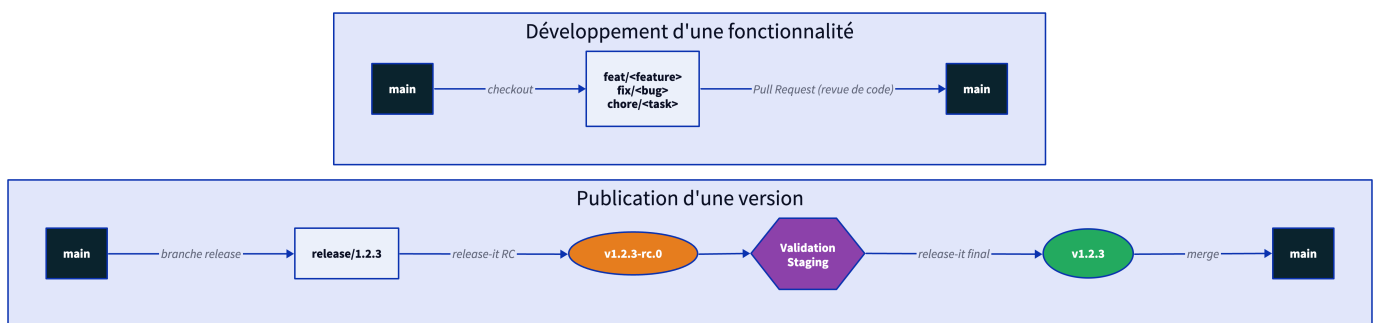


Fig. 2 - Workflow Git : cycle de développement et de publication des versions

Organisation du travail en mode Agile⁶

Méthodologie de développement

L'entreprise a adopté une approche de développement [Agile⁶](#) rythmée par des [Sprints⁶](#) de deux semaines. Bien qu'un framework spécifique comme [Scrum⁶](#) ne soit pas formellement appliqué dans toute sa rigueur, l'organisation du travail s'articule autour de cycles de développement itératifs et de rituels hebdomadaires bien établis.

Parmi ces rituels, on retrouve :

- Le **suivi-services**, qui se tient chaque lundi à 10h30. Cette réunion permet à chaque membre de l'équipe de partager ses avancées et les points de blocage éventuels.
- La **weekly tech**, qui a lieu le vendredi à 10h30. Ce point synchronise l'ensemble des équipes techniques ([Backend⁶](#), [Frontend⁶](#), mobile, data) et assure que chacun est informé des progrès et des défis des autres pôles.

Cette organisation, combinée à des équipes cross-fonctionnelles, permet de livrer de la valeur en continu tout en maintenant une forte cohésion et une bonne circulation de l'information au sein du département technique.

Pipeline CI/CD⁶

Le pipeline de [CI/CD⁶](#) est au cœur de la méthodologie de développement. Il automatise la compilation, les tests et le déploiement du code, garantissant ainsi une haute qualité et une grande vitesse.

Versionnage sémantique

L'équipe de développement suit les conventions du [Semantic Versioning⁶](#) pour gérer les versions de ses applications et services. Cela permet de communiquer clairement l'impact des changements (corrections de bugs, nouvelles fonctionnalités, changements cassants) aux autres équipes et aux utilisateurs de l'[API⁶](#).

Architecture orientée services

L'architecture **microservices** permet de découpler les différentes parties de la plateforme. Chaque service est responsable d'une fonctionnalité métier spécifique et peut être développé, déployé et mis à l'échelle indépendamment des autres. [Kafka⁶](#) joue un rôle crucial en permettant à ces services de communiquer de manière asynchrone et fiable.

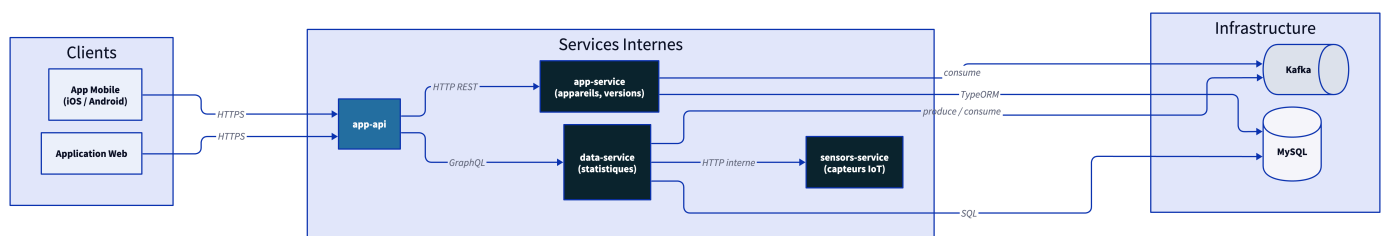


Fig. 3 - Topologie des services internes et de leur communication

Standards de qualité

La qualité est assurée par une combinaison de revues de code systématiques, de tests automatisés (unitaires, intégration) intégrés au pipeline de [CI/CD⁶](#), et d'un monitoring proactif en production. La

robustesse de l'architecture est conçue pour supporter une charge élevée tout en garantissant une haute disponibilité.

Sécurité [Docker](#)

L'utilisation de [Docker](#) suit les meilleures pratiques de sécurité, notamment l'utilisation d'images de base minimalistes et vérifiées, la gestion des secrets en dehors des images, et potentiellement l'analyse des images pour détecter des vulnérabilités connues.

Onboarding et documentation

L'intégration des nouveaux membres est une priorité. Le mentorat par des membres plus expérimentés de l'équipe est une pratique courante. La culture du partage de connaissances est également encouragée, notamment via le blog technique de l'entreprise qui sert de documentation sur les choix d'architecture et les défis techniques rencontrés.

Communication et collaboration

La communication est fluide et directe grâce à la hiérarchie aplatée. Les équipes cross-fonctionnelles travaillent en étroite collaboration au quotidien. Les outils de messagerie instantanée et de gestion de projet viennent supporter ces échanges.

II - Conclusion partielle

Cet environnement de travail est celui d'une [Scale-up](#) technologique mature, qui a su conserver l'agilité et l'esprit d'initiative d'une startup tout en mettant en place des processus robustes pour garantir la qualité, la sécurité et la scalabilité de sa plateforme. La culture d'entreprise, axée sur l'autonomie, la transparence et l'intéressement collectif, constitue un atout majeur pour attirer et retenir les talents.

Missions

I - Optimisation de requête

Contexte et problématique

Le service `stats-service`

`stats-service` est un [Microservice](#) dédié à l'agrégation de données statistiques pour la plateforme Affluences. Il expose une [API](#) [GraphQL](#) construite avec **GraphQL Yoga** et **Type-GraphQL**, et interroge une base MySQL distincte (`stats`) contenant l'historique des mesures de capteurs. Son rôle est de fournir aux tableaux de bord les métriques d'affluence en temps réel et sur des périodes passées : occupations, temps d'attente, entrées et sorties.

La table centrale est `stats.histories`, dont la clé primaire composite est (`measuring_set_id`, `record_datetime_utc`). Chaque ligne représente un relevé horodaté pour un ensemble de capteurs (**measuring set**), et stocke ses valeurs dans une colonne [JSON](#) `data_points` (champs `occupancy`, `waiting_time`, `entries`, `exits`, etc.). Un **measuring set** regroupe les capteurs associés à un site donné ; la relation `site_id -> measuring_set_id` est gérée par le service `sensors-service`.

La requête `getAttendanceStatsForAPeriod`

La requête [GraphQL](#) `getAttendanceStatsForAPeriod` prend en entrée un `siteId` et une plage temporelle (`fromDatetimeUtc`, `toDatetimeUtc`), et retourne les valeurs minimales et maximales d'occupation et de temps d'attente pour cette période. Elle alimente directement les graphiques de synthèse des dashboards clients.

Le flux d'exécution suit la chaîne suivante :



Fig. 4 - Chaîne d'exécution de `getAttendanceStatsForAPeriod`

Cette requête présentait des problèmes de performance critiques pour les plages de dates supérieures à un mois. Les requêtes prenaient plus de 90 secondes pour des périodes de 30 jours et crashaient complètement pour des requêtes sur une année entière.

Symptômes observés :

- Requêtes sur 30 jours : **90+ secondes**
- Requête sur 1 an : **Timeout** (crash complet)
- Dégradation exponentielle avec l'augmentation de la période
- Impact négatif sur l'expérience utilisateur des dashboards

Analyse technique de la cause racine

Le problème résidait dans l'architecture des requêtes du `AttendanceStatsRepository`, qui filtraient les données par `site_id` en utilisant l'extraction [JSON^s](#) :

```
1 WHERE JSON_EXTRACT(h.contextual_data, "$.site_id") = :siteId
2 AND h.record_datetime_utc BETWEEN :from AND :to
3 AND JSON_EXTRACT(h.data_points, "$.occupancy") IS NOT NULL
```

Limitations de l'approche initiale

- Impossibilité d'utiliser l'index de clé primaire (`measuring_set_id`, `record_datetime_utc`)
- Nécessité d'un parcours complet de la table ([Full table scan^s](#))
- Parsing [JSON^s](#) pour chaque ligne de la table
- Performance dégradant de manière exponentielle avec la taille de la période

I.III - Solution architecturale

L'optimisation a consisté à inverser la stratégie de requêtage pour exploiter l'indexation existante de la base de données.

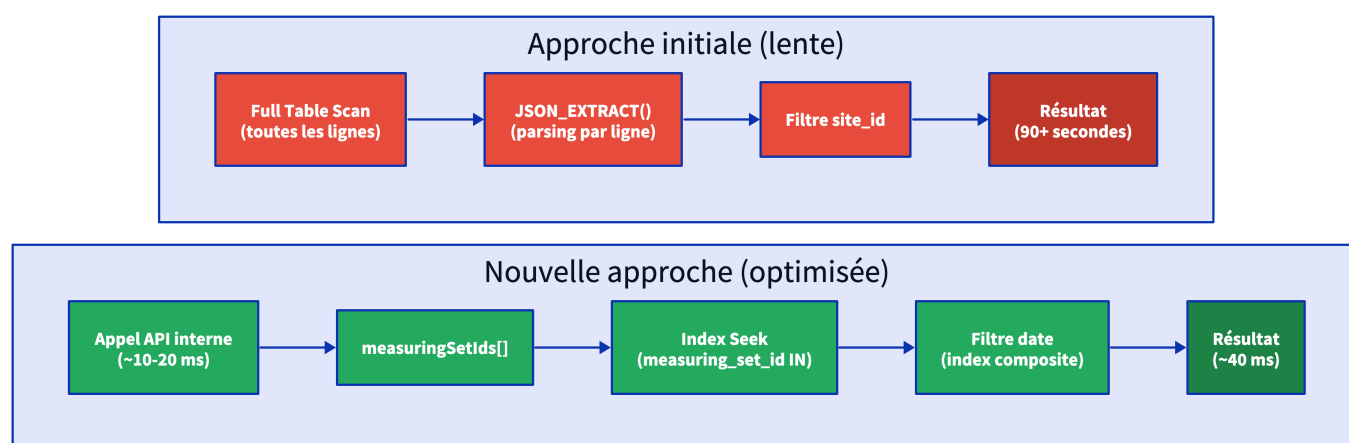


Fig. 5 - Comparaison des plans d'exécution avant et après l'optimisation

Au lieu de requêter directement par `site_id` (stocké dans un champ [JSON^s](#)), la solution procède en deux étapes :

1. **Récupération des measuring set IDs** via le `SensorsInternalHttpRepository`
2. **Requête par `measuring_set_id`** (colonne indexée) au lieu de `site_id` (champ [JSON^s](#))

Cette approche ajoute un appel [API^s](#) léger (10-20ms) mais transforme la requête base de données de $O(n)$ en $O(\log n)$.

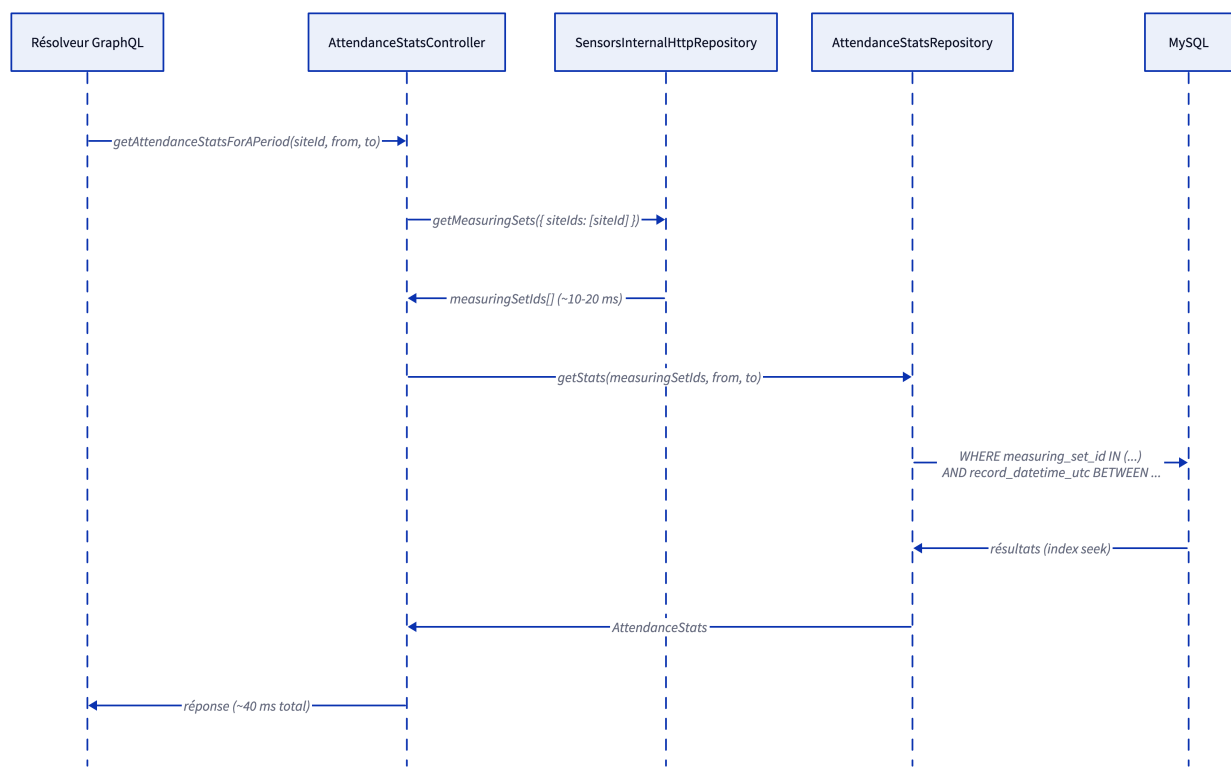


Fig. 6 - Diagramme de séquence de la stratégie d'optimisation en deux étapes

I.IV - Modifications techniques implémentées

Refactoring du contrôleur

Avant ce refactoring, `AttendanceStatsController` était une classe ordinaire sans décorateur, instanciée manuellement à l'intérieur du resolver. Cela impliquait que la `SensorsInternalRepository` devait être construite ou passée explicitement à chaque point d'usage, sans aucune gestion du cycle de vie.

Le passage à un composant géré par le conteneur [IoC](#) **Inversify** s'effectue en trois temps :

1. Le décorateur `@injectable()` signale au conteneur que la classe peut être instanciée et câblée automatiquement.
2. Les dépendances (ici `SensorsInternalRepository`) sont déclarées en paramètre de constructeur ; Inversify les résout seul au démarrage.
3. L'enregistrement `inSingletonScope()` dans `app.module.ts` garantit une unique instance partagée pour toute la durée de vie du service, évitant de recréer le client HTTP à chaque requête.

```

1 @injectable()
2 export class AttendanceStatsController {
3   constructor(private sensorsRepository: SensorsInternalRepository) {}
4
5   private async getMeasuringSetIdsForSite(siteId: number): Promise<string[]> {
6     const measuringSets = await this.sensorsRepository.getMeasuringSets({
7       siteIds: [siteId],
8     });
9
10    return measuringSets.map(ms => ms.measuringSetId);
11  }
12 }
  
```


Optimisation des requêtes repository

Les signatures de méthodes ont été modifiées pour accepter des `measuring_set_ids` :

```
1 public static async getMinMaxOccupancy(
2     measuringSetIds: string[],
3     fromDateUtc: string,
4     toDateUtc: string
5 ): Promise<{ maxOccupancy: number; minOccupancy: number } | null>
```

Les requêtes SQL ont été optimisées pour exploiter l'index :

```
1 WHERE h.measuring_set_id IN (: ... measuringSetIds)
2 AND h.record_datetime_utc BETWEEN :from AND :to
3 AND h.data_points -> "$.occupancy" IS NOT NULL
```

Améliorations techniques

- Utilisation de l'index de clé primaire
- Remplacement de `JSON_EXTRACT()` par l'opérateur `->` (plus lisible)
- Ajout de vérifications de nullité pour les tableaux vides
- Binding TypeORM d'arrays avec la syntaxe `: ...array` pour les clauses `IN`

Injection de dépendances

L'enregistrement dans le conteneur et le câblage avec le resolver sont symétriques :

```
1 // app.module.ts
2 serviceContainer.bind<AttendanceStatsController>(AttendanceStatsController)
3     .toSelf()
4     .inSingletonScope();
5
6 // AttendanceStatsResolver.ts
7 @injectable()
8 export class AttendanceStatsResolver {
9     constructor(private attendanceStatsController: AttendanceStatsController) {}
10 }
```

Le resolver déclare simplement avoir besoin d'un `AttendanceStatsController` ; le conteneur se charge de lui fournir l'instance singleton déjà câblée avec sa `SensorsInternalRepository`. Aucune des deux classes ne sait comment l'autre est construite.

Ce découplage présente un avantage concret pour les tests unitaires : il suffit de lier `SensorsInternalRepository` à une implémentation fictive dans le conteneur de test pour isoler entièrement la logique du controller, sans modifier une ligne de code de production.

Résultats et impact

- Gains de performance mesurés

Métrique	Avant	Après	Amélioration
Requête sur 30 jours	90+ secondes	40 ms	×2 250
Requête sur 1 an	Crash (timeout)	220 ms	∞
Opération DB	Full table scan^a	Index seek^a	-
Comportement	Dégradation exp.	Performance linéaire	-

Explication des optimisations clés

Exploitation de l'index composite

La table `stats.histories` possède une clé primaire composite (`measuring_set_id`, `record_datetime_utc`). En base de données relationnelle, une clé primaire est automatiquement indexée via un [Arbre B \(B-Tree\)](#)[§]. Cet index peut être vu comme un annuaire trié : chercher une entrée revient à naviguer dans l'arbre plutôt qu'à feuilleter toutes les pages. La complexité passe de $O(n)$ (lire toutes les lignes) à $O(\log n)$ (descendre l'arbre).

Pour qu'un index composite soit utilisé, la requête doit filtrer en commençant par la **première colonne** de l'index. Ici, `measuring_set_id IN (...)` satisfait cette condition : MySQL identifie directement les feuilles de l'arbre correspondant aux measuring sets demandés. La clause `record_datetime_utc BETWEEN` exploite ensuite la **seconde colonne** pour affiner la plage temporelle à l'intérieur de chaque partition de measuring set.

Le problème de la requête d'origine était précisément que `JSON_EXTRACT(h.contextual_data, "$.site_id")` n'est pas une colonne, mais une expression calculée. MySQL ne peut pas indexer une expression dynamique sans colonne générée explicite, et doit donc évaluer cette extraction pour **chaque ligne de la table** avant de filtrer : c'est le [Full table scan](#)[§].

Definition 4.2. ([Full table scan](#)[§] vs [Index seek](#)[§]):

- **Full table scan**[§] : MySQL lit séquentiellement toutes les pages disque de la table pour trouver les lignes correspondantes. Sur une table de plusieurs millions de lignes, cela représente des gigaoctets de lecture, indépendamment du nombre de résultats attendus.
- **Index seek**[§] : MySQL descend le [B-Tree](#)[§] de l'index et accède directement aux pages pertinentes. Seules les données nécessaires sont lues. Le coût est proportionnel au nombre de résultats, pas à la taille totale de la table.

Trade-off et analyse coût-bénéfice

La stratégie en deux étapes introduit un appel réseau supplémentaire, ce qui peut sembler contre-intuitif. L'analyse chiffrée justifie ce choix :

Analyse coût-bénéfice

Operation	Avant	Après
Appel HTTP <code>getMeasuringSets</code>	absent	10-20 ms
Requête SQL (30 jours)	90+ secondes	20-30 ms
Requête SQL (1 an)	timeout	200 ms
Total (1 an)	crash	220 ms

L'appel HTTP vers `sensors-service` retourne une liste de quelques dizaines d'identifiants : la réponse est petite, le service est interne au réseau privé, et la latence est négligeable au regard des 90 secondes économisées. C'est un coût fixe et prévisible, indépendant de la période interrogée.

A l'inverse, le parsing JSON ligne par ligne croissait linéairement avec le volume de données : plus la période était longue, plus la table était parcourue en entier, et plus le temps d'exécution explosait. Pour une requête annuelle, la table entière devait être lue, parsee et filtrée, saturant à la fois le CPU du serveur MySQL et ses I/O disque.

Impact en production

Cette optimisation a permis de :

- Rendre les dashboards réactifs pour des statistiques annuelles en temps réel
- Réduire drastiquement la charge sur la base de données
- Éliminer les erreurs de timeout pour les requêtes multi-mois
- Améliorer significativement l'expérience utilisateur avec des réponses instantanées

Conformité aux patterns existants

L'optimisation suit le même pattern déjà utilisé avec succès dans :

- `app/modules/attendance/infrastructure/record-history-mysql.repository.ts`
- `app/modules/attendance/services/attendance.service.ts`

La solution réutilise l'infrastructure existante (`SensorsInternalHttpRepository`) et respecte le pattern repository utilisé dans l'ensemble du codebase.

Enseignements techniques

Leçons clés

1. **Conscience des index** : Toujours concevoir les requêtes autour des index disponibles
2. **Prudence avec les colonnes `JSON`** : Le filtrage sur des champs `JSON` empêche l'utilisation d'index
3. **Requêtes en deux étapes** : Ajouter une étape de lookup légère peut être plus rapide qu'une requête unique non optimisée
4. **Mesurer systématiquement** : L'amélioration de `x2 250` a été validée par des mesures en production réelle
5. **Suivre les patterns existants** : La solution réutilise l'architecture établie du projet

Concepts techniques approfondis

- **Index composites** : Compréhension du fonctionnement de `(measuring_set_id, record_datetime_utc)`
- **TypeORM array binding** : Syntaxe `:...array` pour les clauses `IN`
- **Opérateurs `JSON` MySQL** : Utilisation de `->` au lieu de `JSON_EXTRACT()`
- **DI** : Patterns `IoC` pour améliorer testabilité et maintenabilité

Date de réalisation : Octobre 2025

Statut : [OK] Déployé en production et valide avec du trafic réel

II - Création du `app-service`

Contexte et objectifs

Dans le cadre de la gestion des applications mobiles de la société, la plateforme repose sur des **millions d'appareils** enregistrés (smartphones iOS et Android). Ces appareils communiquent avec le backend via une clé [API](#) qui sert à les authentifier et à les autoriser. Jusqu'alors, la logique de gestion de ces appareils était dispersée dans d'autres services.

J'ai eu comme objectif de créer un nouveau [Microservice](#) dédié, nommé `app-service`, afin de centraliser tout ce qui touche à la gestion des appareils et des versions applicatives. Ce service est destiné à être consommé principalement par `app-api`.

Les responsabilités attendues étaient :

- Enregistrement d'un nouvel appareil.
- Mise à jour des informations d'un appareil (token Firebase, dernière version, etc.).
- Vérification de l'autorisation d'un appareil (clé [API](#) valide, liste noire).
- Gestion de l'historique des versions de l'application.

Cette première version avait pour périmètre la mise en place de la **base du projet** : connexion à la base de données, définition des entités, et exposition d'une [API](#) REST.

Contraintes techniques

La contrainte principale était l'**échelle** : la table `psn.appareils` contient des millions de lignes. Toute décision d'architecture (indexation, pagination, requêtes) devait tenir compte de cette volumétrie.

La stack choisie est **NestJS** avec **TypeORM** pour l'accès à la base **MySQL**, en suivant les conventions du projet (`@affluences/commons`). Le service expose une [API](#) REST versionnée (v1).

Architecture mise en place

Le service suit le pattern **Clean Architecture** adopté en interne, avec une séparation claire entre :

- `core/` : interfaces/abstractions (contrats de repository)
- `modules/<feature>/entities/` : entités TypeORM (mapping base de données)
- `modules/<feature>/models/` : modèles métier et DTOs de l'[API](#)
- `modules/<feature>/adapters/` : conversion entité ↔ modèle
- `modules/<feature>/repositories/` : implémentation MySQL du repository
- `modules/<feature>/services/` : logique métier
- `modules/<feature>/controllers/` : exposition REST

Deux modules ont été créés : `DevicesModule` et `AppVersionsModule`.

Entités et tables

`DeviceEntity` : table `psn.appareils`

L'entité représente un appareil enregistré :

```
1 @Entity('appareils', { schema: 'psn', database: 'psn' })
2 @Unique('cle_UNIQUE', ['apiKey'])
3 @Unique('identifieur_UNIQUE', ['identifieur', 'revokedAt'])
4 export class DeviceEntity {
```

```

5  @PrimaryGeneratedColumn({ type: 'int', name: 'appareil_id' })
6  deviceId!: number;
7
8  @Column('varchar', { name: 'identifieur', nullable: false, length: 255 })
9  identifieur!: string;
10
11 @Column('varchar', { name: 'cle', nullable: false, length: 255, unique: true })
12 apiKey!: string;
13
14 @Column('varchar', { name: 'operating_system', ... })
15 operatingSystem!: string;
16
17 @Column('tinyint', { name: 'blacklist', ...
18   transformer: new BoolTinyIntTransformer() })
19 blacklist!: boolean;
20
21 @Column('varchar', { name: 'firebase_token', nullable: true, length: 255 })
22 firebaseToken?: string | null;
23
24 @Column('datetime', { name: 'revoked_at', nullable: true })
25 revokedAt?: Date | null;
26
27 @Column('enum', { name: 'revoked_reason', enum: RevokedReason, nullable: true })
28 revokedReason?: RevokedReason | null;
29 // ...
30 }

```

Deux contraintes d'unicité garantissent l'intégrité : la clé [API](#)^a est globalement unique, et l'identifiant d'appareil est unique parmi les appareils non révoqués (la combinaison `identifieur + revokedAt` est unique, ce qui permet d'avoir plusieurs entrées historiques révoquées pour un même identifiant).

AppVersionEntity : **table** `psn.app_version_history`

L'entité représente une version de l'application mobile :

```

1  @Entity('app_version_history', { schema: 'psn', database: 'psn' })
2  @Unique('build_UNIQUE', ['build', 'appIdentifier'])
3  export class AppVersionEntity {
4    @PrimaryGeneratedColumn({ type: 'int', name: 'app_version_history_id' })
5    appVersionHistoryId!: number;
6
7    @Column('varchar', { name: 'app_identifieur' })
8    appIdentifier!: string;
9
10   @Column('varchar', { name: 'version_identifieur' })
11   versionIdentifier!: string;
12
13   @Column('double', { name: 'build', default: 0 })
14   build!: number;
15
16   @Column('bit', { name: 'expired', transformer: new BoolBitTransformer() })
17   expired!: boolean;
18
19   @Column('bit', { name: 'available', transformer: new BoolBitTransformer() })
20   available!: boolean;
21
22   @Column('bit', { name: 'beta_build', transformer: new BoolBitTransformer() })
23   betaBuild!: boolean;
24 }

```

psn.appareils			psn.app_version_history		
appareil_id	INT	PK	app_version_history_id	INT	PK
identifieur	VARCHAR(255)		app_identifieur	VARCHAR(100)	
cle	VARCHAR(255)	UNQ	version_identifieur	VARCHAR(50)	
operating_system	VARCHAR(50)		build	DOUBLE	
blacklist	TINYINT(1)		expired	BIT(1)	
firebase_token	VARCHAR(255) NULL		available	BIT(1)	
revoked_at	DATETIME NULL		beta_build	BIT(1)	
revoked_reason	ENUM NULL				

Fig. 7 - Schéma des tables gérées par `app-service`

Endpoints REST exposés

- Devices : `/v1/devices`

Méthode	Chemin	Description
GET	<code>/v1/devices</code>	Liste paginée avec filtres (deviceId, apiKey, identifier, isRevoked)
GET	<code>/v1/devices/:deviceId</code>	Récupération d'un appareil par son ID
POST	<code>/v1/devices</code>	Création d'un nouvel appareil
PATCH	<code>/v1/devices/:deviceId</code>	Mise à jour partielle d'un appareil

- App Versions : `/v1/app-versions`

Méthode	Chemin	Description
GET	<code>/v1/app-versions</code>	Liste paginée des versions avec filtres (type, version, build, available)

La pagination est gérée via le package partagé `@affluences/commons/pagination` qui expose un objet `Paginated<T>`.

Points techniques notables

Gestion de la révocation

Un appareil peut être révoqué (banni) sans être supprimé physiquement. Les champs `revokedAt` et `revokedReason` permettent de tracer la révocation tout en conservant l'historique. La contrainte d'unicité sur `(identifieur, revokedAt)` permet d'avoir plusieurs entrées pour un même identifiant physique (un téléphone réinstallant l'application), tant qu'une seule n'est pas révoquée.

Transformateurs de types MySQL

La base de données utilise des types `TINYINT` et `BIT` pour les booléens. Des transformateurs TypeORM (`BoolTinyIntTransformer`, `BoolBitTransformer`) assurent la conversion transparente vers des `boolean` TypeScript, évitant les erreurs de type à l'échelle de millions de lignes.

Bootstrap et middlewares

Le service est initialisé avec un ensemble de middlewares communs issus de `@affluences/commons/router` :

- Compression des réponses
- Parsing des vraies IPs (proxies)
- Injection d'un identifiant de requête (`requestId`)
- Exposition de métriques Prometheus (`/metrics`)

L'**OpenTelemetry** (OTel) est initialisée **avant** l'import de NestJS, conformément aux exigences de l'instrumentation automatique.

Résultat

Le service `app-service` a été mergé et déployé en environnement d'intégration et de staging. Il constitue le socle sur lequel les fonctionnalités de vérification d'autorisation des appareils seront construites dans les prochains tickets.

Date de réalisation : Octobre à Novembre 2025

Statut : [OK] Mergé sur `main`, `staging` et en production

Annexes

I - Rona : automatiser les commits

I.I - Présentation

En parallèle de mon travail chez Affluences, j'ai développé et maintenu [Rona](#), un outil en ligne de commande écrit en **Rust** dont l'objectif est de rationaliser le workflow Git quotidien. Le projet est open source, publié sur [crates.io](#) et distribué via Homebrew.

Rona a été conçu avant mon arrivée chez Affluences pour simplifier les opérations Git répétitives. Cependant, c'est l'expérience au sein de l'équipe **Internal Services** (avec ses conventions strictes de commits, ses tickets Jira et son hook **commitlint**) qui a véritablement motivé le développement de la fonctionnalité centrale : le système de champs personnalisables dans `.rona.toml`.

I.II - Fonctionnalités principales

Fonctionnalités de Rona

- **Staging intelligent** (`rona -a`) : ajout de fichiers au staging avec exclusion de patterns, fonctionnel depuis n'importe quel sous-répertoire du dépôt.
- **Génération de message de commit** (`rona -g`) : sélection interactive du type de commit, puis ouverture de l'éditeur configuré ou saisie directe en terminal (`-i`).
- **Commit et push** (`rona -c -p`) : commit depuis le fichier `commit_message.md`, avec détection automatique de la signature GPG.
- **Synchronisation de branche** (`rona sync`) : mise à jour d'une branche depuis `main` par merge ou rebase.
- **Complétions shell** : support Bash, Fish, Zsh et PowerShell.

I.III - Système de configuration

Rona suit une hiérarchie de configuration à deux niveaux :

- `~/.config/rona.toml` : configuration globale, appliquée à tous les projets.
- `.rona.toml` : configuration locale au projet, qui prend la priorité sur la configuration globale.

La clé `template` permet de définir le format exact du message de commit via des variables (`{commit_type}`, `{message}`, `{branch_name}`, etc.) et des blocs conditionnels (`{?ticket}...{/ticket}`) pour gérer les champs optionnels.

La fonctionnalité `[[extra_fields]]` permet de déclarer des champs supplémentaires affichés lors de la génération interactive. Chaque champ peut être pré-rempli automatiquement depuis la sortie d'une commande shell ou depuis le nom de la branche courante grâce à une expression régulière.

I.IV - Configuration adaptée à Affluences

La partie `[[extra_fields]]` a été conçue en grande partie pour répondre aux besoins d'Affluences : extraire automatiquement le numéro de ticket `INT-XXXX` depuis le nom de la branche, et suggérer

le scope depuis l'historique git récent. Le `.rona.toml` suivant produit des commits conformes au format imposé par le `.commitlintrc.json` de l'équipe :

```
1 editor = "zed"
2 commit_types = ["feat", "fix", "docs", "refactor", "test", "revert", "quality", "chore", "init"]
3
4 # Produit: feat(devices): add PATCH endpoint #INT-1234
5 template = "{commit_type}{?scope}({scope}){/scope}: {message}{?ticket} #{ticket}{/ticket}"
6
7 field_order = ["scope", "message", "ticket"]
8
9 # Scope suggéré depuis les 20 derniers commits
10 [[extra_fields]]
11 name = "scope"
12 prompt = "Scope"
13 kind = "select"
14 required = true
15 prefetch.source = "command"
16 prefetch.command = "git log -20 --pretty=format:%s"
17 prefetch.extract_regex = "\\w+\\((?P<value>[^\n]*)\\):"
18 prefetch.deduplicate = true
19
20 # Ticket extrait automatiquement du nom de branche (ex: feat/INT-1234_app-service)
21 [[extra_fields]]
22 name = "ticket"
23 prompt = "Ticket Jira"
24 kind = "text"
25 required = false
26 validation = "^[A-Z]+-[0-9]+$"
27 prefetch.source = "branch"
28 prefetch.extract_regex = "[A-Z]+-[0-9]+"
```

Sur une branche nommée `feat/app-service_first-version_INT-3713`, une session `rona -g -i` se déroule ainsi :

```
1 $ Select commit type
2 > feat
3
4 $ Scope
5 > devices
6   app-versions
7   (none)
8   Other (enter manually)
9
10 $ Message
11 > add PATCH endpoint for partial device update
12
13 $ Ticket Jira (INT-3713)
14 > INT-3713
```

Ce qui produit directement :

```
1 feat(devices): add PATCH endpoint for partial device update #INT-3713
```

Le message est conforme au pattern de la `headerPattern` du `.commitlintrc.json` et sera accepté par le hook pre-commit sans modification manuelle.

II - Rédaction avec Typst et `clean-cnam-template`

II.I - Typst, une alternative moderne à LaTeX

Ce mémoire n'a pas été rédigé avec un traitement de texte classique ni avec LaTeX, mais avec Typst, un système de composition de documents de nouvelle génération. Typst adopte une syntaxe légère et expressive, une compilation quasi-instantanée, et un système de packages communautaire, ce qui en fait une alternative pragmatique à LaTeX pour la rédaction de documents techniques et académiques.

II.II - Un package open source dédié au CNAM

Pour structurer mes documents de cours au CNAM, j'ai développé et publié le package open source `clean-cnam-template`, disponible dans le registre officiel de packages Typst sous l'identifiant `@preview/clean-cnam-template`. Ce mémoire lui-même l'utilise en version 1.6.7 :

```
1 #import "@preview/clean-cnam-template:1.6.7": *
```

Le package est conçu de façon modulaire, avec une séparation claire entre la configuration, les composants d'interface, la gestion des polices, la mise en page et les environnements mathématiques. Ses fonctionnalités principales sont :

Fonctionnalités de `clean-cnam-template`

- **Identité visuelle CNAM** : couleurs officielles, logo, en-têtes contextuels.
- **Page de couverture configurable** : titre, sous-titre, auteur (avec lien ORCID et mailto), logo secondaire, couleur de fond, cercles décoratifs : tout est paramétrable.
- **Blocs de code enrichis** (`#code()`) : coloration syntaxique, numérotation des lignes, étiquette de fichier ou de langage.
- **Environnements mathématiques** : `#definition()`, `#example()`, `#theorem()` avec styles distincts.
- **Blocs de contenu** : `#my-block()`, `#blockquote()` pour les encadrés et citations.
- **Plan personnalisable** : le paramètre `outline-code` accepte n'importe quel contenu Typst ; ce mémoire injecte ainsi l'arbre ASCII défini dans `custom-outline.typ`.
- **Gestion typographique avancée** : polices distinctes pour le corps, les titres et le code, mise en évidence automatique de mots-clés avec la couleur principale.

II.III - Usage au quotidien et adoption par les camarades

Le package est utilisé pour l'ensemble de mes prises de notes et rapports de cours au CNAM. Sa cohérence visuelle et sa facilité de configuration (les paramètres couvrent polices, couleurs, dates, auteur et mise en page en un seul bloc `#show: clean-cnam-template.with(...)`) ont conduit plusieurs camarades de promotion à l'adopter pour leurs propres documents.

Ce mémoire constitue lui-même un cas d'usage avancé du template : la table des matières en arbre ASCII, les blocs de code avec la police **Monospace Krypton**, les schémas D2 intégrés comme figures sont autant de personnalisations réalisées par-dessus le template de base.

Glossaire

Ce glossaire regroupe les termes techniques utilisés dans ce document, classés par catégorie pour faciliter la compréhension.

I - Développement

Terme	Définition
Backend	Partie d'une application qui s'exécute sur le serveur. Elle gère la logique métier, les bases de données et les communications avec d'autres services. Invisible pour l'utilisateur final, c'est le "moteur" de l'application.
Frontend	Partie visible d'une application avec laquelle l'utilisateur interagit directement (boutons, formulaires, affichage). C'est l'interface graphique.
API : Application Programming Interface	Interface permettant à deux applications de communiquer entre elles. Par exemple, une application mobile utilise une API pour récupérer des données depuis un serveur.
GraphQL	Langage de requête pour les API, permettant au client de demander exactement les données dont il a besoin, ni plus ni moins. Alternative moderne aux API REST traditionnelles.
Microservice	Approche de développement où une application est décomposée en petits services indépendants, chacun responsable d'une fonction spécifique. Cela facilite la maintenance et permet de faire évoluer chaque partie séparément.
TypeScript	Langage de programmation basé sur JavaScript qui ajoute un système de types. Cela permet de détecter des erreurs avant l'exécution du programme et d'améliorer la qualité du code.
Node.js	Environnement d'exécution permettant d'utiliser JavaScript côté serveur (backend). Très utilisé pour créer des applications web rapides et scalables.
NestJS	Framework backend moderne pour Node.js utilisant TypeScript. Il suit les principes d'architecture modulaire et d'injection de dépendances, facilitant la création d'applications serveur robustes et maintenables.
Flutter	Framework de Google permettant de développer des applications mobiles pour iOS et Android avec un seul code source. Utilise le langage Dart.
Angular	Framework JavaScript/TypeScript développé par Google pour créer des applications web interactives et structurées.

Terme	Definition
IoC : Inversion of Control	Principe de conception logicielle ou le controle du flux d'execution est delegue a un framework. Cela rend le code plus modulaire et plus facile a tester.
DI : Dependency Injection	Technique de programmation ou les dependances d'un composant lui sont fournies de l'exterieur plutot que creees en interne. Facilite les tests et la maintenance.
ORM : Object-Relational Mapping	Technique qui fait le lien entre les objets d'un langage de programmation et les tables d'une base de donnees relationnelle. Permet de manipuler les donnees sous forme d'objets sans ecrire de SQL manuellement.
DTO : Data Transfer Object	Objet dont le seul role est de transporter des donnees entre les couches d'une application ou entre services. Il ne contient pas de logique metier et definit le contrat de l'interface (requete ou reponse d'API).

II - Infrastructure

Terme	Definition
CI/CD : Integration Continue / Deploiement Continu	Pratique d'automatisation qui permet de tester et deployer automatiquement le code a chaque modification. Cela reduit les erreurs humaines et accelere la mise en production des nouvelles fonctionalites.
Docker	Technologie de conteneurisation qui permet d'empaqueter une application avec toutes ses dependances dans un "conteneur" isole. Cela garantit que l'application fonctionnera de la meme maniere sur n'importe quel serveur.
Kubernetes	Systeme d'orchestration de conteneurs qui automatise le deploiement, la mise a l'echelle et la gestion d'applications conteneurisees.
Cloud : Cloud Computing	Modele de fourniture de ressources informatiques (serveurs, stockage, bases de donnees) via Internet, sans avoir a gerer physiquement les machines.
Redis	Base de donnees en memoire tres rapide, souvent utilisee comme cache pour stocker temporairement des donnees frequemment consultees et accelerer les reponses.
Kafka : Apache Kafka	Plateforme de streaming distribuee qui permet de publier et s'abonner a des flux de donnees en temps reel. Utilisee pour construire des pipelines de donnees en temps reel et des applications de streaming.

Terme	Definition
Airflow : Apache Airflow	Plateforme open-source de gestion de flux de travail (workflow) qui permet de definir, planifier et surveiller des pipelines de donnees complexes. Automatise les taches de traitement de donnees.
Argo Workflow	Outil d'orchestration de workflows natif pour Kubernetes. Permet d'executer des taches complexes et des processus batch dans des conteneurs, facilitant l'automatisation et la gestion des pipelines de traitement.

III - Donnees

Terme	Definition
IoT : Internet of Things	Ensemble d'objets physiques connectes a Internet capables de collecter et transmettre des donnees. Chez Affluences, ce sont les capteurs qui mesurent l'affluence en temps reel.
RGPD : Reglement General sur la Protection des Donnees	Reglementation europeenne qui encadre le traitement des donnees personnelles. Elle impose des regles strictes sur la collecte, le stockage et l'utilisation des donnees des utilisateurs.
SQL : Structured Query Language	Langage standard pour interroger et manipuler des bases de donnees relationnelles. Permet de creer, lire, modifier et supprimer des donnees.
NoSQL : Not Only SQL	Type de base de donnees qui ne suit pas le modele relationnel classique. Adapte pour stocker de grands volumes de donnees non structurees ou semi-structurees.
JSON : JavaScript Object Notation	Format de donnees textuel leger et lisible, tres utilise pour echanger des informations entre applications, notamment via les API.
B-Tree : Arbre B	Structure de donnees arborescente utilisee par les moteurs de bases de donnees pour organiser les index. Permet de localiser une entree en $O(\log n)$ operations plutot qu'en $O(n)$, quelle que soit la taille de la table.
Full table scan	Operation de base de donnees ou le moteur lit sequentiellement toutes les lignes d'une table pour trouver celles qui correspondent a un filtre. Tres couteuse sur les grandes tables car le cout croit lineairement avec le volume de donnees.
Index seek	Operation de base de donnees ou le moteur utilise un index (B-Tree) pour acceder directement aux lignes pertinentes, sans lire l'ensemble de la table. Le cout est proportionnel au nombre de resultats, pas a la taille totale de la table.

IV - Methodologie

Terme	Definition
Agile	Approche de gestion de projet qui privilegie les cycles courts, l'adaptation au changement et la collaboration. Le travail est divise en petites iterations permettant de livrer regulierement de la valeur.
Sprint	Periode de travail courte et fixe (generalement 1 a 4 semaines) pendant laquelle une equipe s'engage a realiser un ensemble de taches definies. A la fin du sprint, un produit fonctionnel est livre.
Scrum	Framework agile populaire organisant le travail en sprints avec des roles definis (Product Owner, Scrum Master, Equipe) et des ceremonies regulieres (daily meeting, retrospective).
SemVer : Semantic Versioning	Convention de numerotation des versions logicielles au format MAJEUR.MINEUR.CORRECTIF. Permet de comprendre rapidement l'impact d'une mise a jour.
Monorepo : Monolithic Repository	Architecture de gestion de code ou plusieurs projets ou services sont stockes dans un seul et meme depot. Facilite le partage de code, la gestion des dependances communes et la coordination des modifications entre projets.

V - Securite

Terme	Definition
ISO 27001 : ISO/IEC 27001	Norme internationale de reference pour la gestion de la securite de l'information. La certification atteste qu'une organisation a mis en place des processus robustes pour proteger ses donnees.
SSL/TLS : Secure Sockets Layer / Transport Layer Security	Protocoles de securite qui chiffrent les communications sur Internet. C'est ce qui permet d'avoir des connexions securisees (le cadenas dans la barre d'adresse du navigateur).
Sentry	Outil de monitoring qui detecte et signale automatiquement les erreurs dans les applications en production, permettant aux developpeurs de les corriger rapidement.
APM : Application Performance Monitoring	Outils de surveillance des performances des applications qui permettent d'identifier les lenteurs, les goulots d'etranglement et d'optimiser les temps de reponse.
Datadog	Plateforme de monitoring et d'observabilite cloud. Permet de surveiller les performances des applications, analyser les logs, detecter les anomalies et visualiser l'etat de l'infrastructure en temps reel.

VI - Entreprise

Terme	Definition
Scale-up	Entreprise en forte croissance qui a depasse le stade de startup. Elle a valide son modele economique et cherche a se developper rapidement tout en structurant son organisation.